Golden Implementation Prompt

Sprint 1: The Core Loop

Authentic AI Voice MVP • Type → Save → Export PDF

1. Role & Context

You are an expert Full-Stack Developer (Next.js 14 + FastAPI + PostgreSQL). You are implementing **Sprint 1** of the "Authentic AI Voice" web application — an AI-powered writing tool that learns a user's writing style.

This sprint delivers a single, end-to-end vertical slice: **a user can create a document, write rich formatted text in a Tiptap editor, auto-save it to PostgreSQL, and download it as a PDF.**

Security Context: There is **no authentication** in this sprint. All endpoints are open and `user_id` is hardcoded to `"dev-user"` in the API — this is intentional and will be replaced in Sprint 2.

2. Tech Stack (Strictly Follow This)

Layer	Technology
Backend Framework	FastAPI 0.110.3 with Python 3.12
Database	PostgreSQL 18 via SQLAlchemy 2.0 async + asyncpg
ORM / Migrations	SQLAlchemy `mapped_column` + Alembic
PDF Export	WeasyPrint 62.3 (lazy import — GTK3 may not be installed)
Frontend Framework	Next.js 14.2.35 App Router
Editor	Tiptap **v3.22.5** (NOT v2 — API differs)
State Management	Zustand 5.0.13
HTTP Client	Axios (already configured in `lib/api.ts`)
Styling	Tailwind CSS 3.4.19
Language	TypeScript 5.9.3

3. Project Structure (Already Exists)

```
project-root/
|-- backend/
|   |-- .env                          <- already configured
|   |-- requirements.txt              <- already installed
|   |-- alembic/                      <- already configured
|   `-- app/
|       |-- main.py                   <- already written (needs NO
    changes)
|       |-- models/                   <- user.py, document.py already
    written
|       |-- routers/
|       |   |-- documents.py          <- MODIFY: remove auth dependency
|       |   `-- export.py             <- MODIFY: remove auth dependency
|       |-- services/                 <- pdf_service.py already written
|       `-- uploads/                  <- samples/ and exports/
`-- frontend/
    |-- package.json                  <- already installed
    |-- lib/api.ts                    <- already written
    |-- store/editorStore.ts          <- already written
    |-- components/
```

```
|   `-- editor/
|       |-- TiptapEditor.tsx      <- BUILD THIS
|       `-- Toolbar.tsx           <- BUILD THIS
`-- app/
    |-- globals.css               <- MODIFY: add design system
    |-- dashboard/page.tsx        <- BUILD THIS
    `-- documents/[id]/page.tsx   <- BUILD THIS
```

4. Backend: Changes Required for Sprint 1

Task B1: Remove auth guards from documents and export routers

Replace all user references with a hardcoded `DEV_USER_ID = "dev-user"`. **Preferred approach — seed a dev user on startup** in `main.py`:

```python
# In main.py, add a startup event:
from sqlalchemy import select
from app.database import AsyncSessionLocal
from app.models.user import User
from passlib.context import CryptContext

@app.on_event("startup")
async def seed_dev_user():
    async with AsyncSessionLocal() as db:
        existing = await db.get(User, "dev-user")
        if not existing:
            dev = User(
                id="dev-user",
                email="dev@localhost",
                hashed_password=CryptContext(schemes=["bcrypt"]).hash("
                    devpass"),
            )
            db.add(dev)
            await db.commit()
```

Then update `routers/documents.py` and `routers/export.py`: Remove `user: User = Depends(get_current_user)` from all endpoints and replace `user.id` with the constant `"dev-user"` everywhere.

Task B2: Verify PDF export endpoint works

`POST /api/export/pdf` must: Accept `{ "document_id": "<uuid>" }`, fetch document, convert JSON → HTML, render with WeasyPrint, save to `uploads/exports/`, and return `{ "filename": "...", "download_url": "..." }`.

5. Frontend: Full Implementation Required

Task F1: Design System (`app/globals.css`)

Directive: Moody, dark-mode aesthetic with neon accents and heavy glassmorphism. Do not use default flat Tailwind colors.

```css
:root {
  --color-bg-deep: #050505;          /* Pitch/abyss black */
  --color-glass: rgba(255, 255, 255, 0.03);
  --color-neon-primary: #06b6d4;     /* Electric Cyan */
```

```
  --color-neon-secondary: #8b5cf6; /* Glowing Purple */
}
```

Task F2: Dashboard Page (`app/dashboard/page.tsx`)

- **On mount:** `GET /api/documents` via `api.get(...)`. Show 3 animated loading skeleton cards while fetching.
- **Grid:** Render document cards (title, word count, last updated). Add hover elevation.
- **Action:** "New Document" button calls `POST /api/documents` then routes to editor.

Task F3 & F4: Tiptap Editor & Toolbar (`components/editor/`)

- Use `useEditor` from `@tiptap/react` (v3).
- **Extensions:** StarterKit, Underline, Link, Image, CodeBlockLowlight, Table family.
- **Toolbar:** Groups for Format (B/I/U/S), Headings, Lists/Blocks, Tables, and Save Status.

Task F5: Document Editor Page (`app/documents/[id]/page.tsx`)

```
+-----------------------------------------------------+
|  <- Back   [Editable Title]          [Export PDF]   |
+-----------------------------------------------------+
|  B I U S | H1 H2 H3 | Bullet List | Save Status     |
+-----------------------------------------------------+
|              Tiptap Editor Content Area             |
+-----------------------------------------------------+
```

Wire `onUpdate` → `useAutoSave.scheduleSave`. Export flow must force save, POST to export endpoint, open PDF in new tab, and trigger a success toast.

6. API Reference (Sprint 1 Endpoints)

Method	Path	Body	Response
GET	`/api/documents`	—	`DocumentOut[]`
POST	`/api/documents`	`{ title: string }`	`DocumentDetail`
GET	`/api/documents/{id}`	—	`DocumentDetail`
PATCH	`/api/documents/{id}`	`{ title?, content? }`	`DocumentOut`
POST	`/api/export/pdf`	`{ document_id: str }`	`{ url, filename }`

7. Definition of Done — Sprint 1

Backend

- ☐ `GET /api/documents` returns empty array on fresh start.
- ☐ `POST /api/documents` creates a document with `user_id = "dev-user"`.
- ☐ `POST /api/export/pdf` returns a `download_url` and the file exists at that URL.
- ☐ Swagger UI at `http://localhost:8000/docs` shows all routes.

Frontend

- ☐ `http://localhost:3000` redirects to `/dashboard`.
- ☐ Dashboard shows a loading skeleton, then the list of documents.
- ☐ Editor auto-saves 3 seconds after last keystroke. Status shows "Saving..." → "Saved ✔".
- ☐ Title is editable; saves on blur.
- ☐ "Export PDF" button saves → POSTs to API → opens PDF in new tab.
- ☐ Design is dark, premium, and uses Inter font — not the default Next.js template.

8. Critical Implementation Notes

1. **Tiptap v3 imports** — All extensions import from their own packages. Do NOT use `@tiptap/core` directly.
2. `"use client"` **directive** — Required on every file that uses React hooks.
3. **Auto-save JSON** — Send `editor.getJSON()` (an object), NOT `editor.getHTML()` (a string).
4. **Missing GTK3** — If WeasyPrint fails (503), show user toast: *"PDF export requires GTK3"*.
5. **Initial Content** — New docs seed `content: { "type": "doc", "content": [{ "type": "paragraph" }] }`.